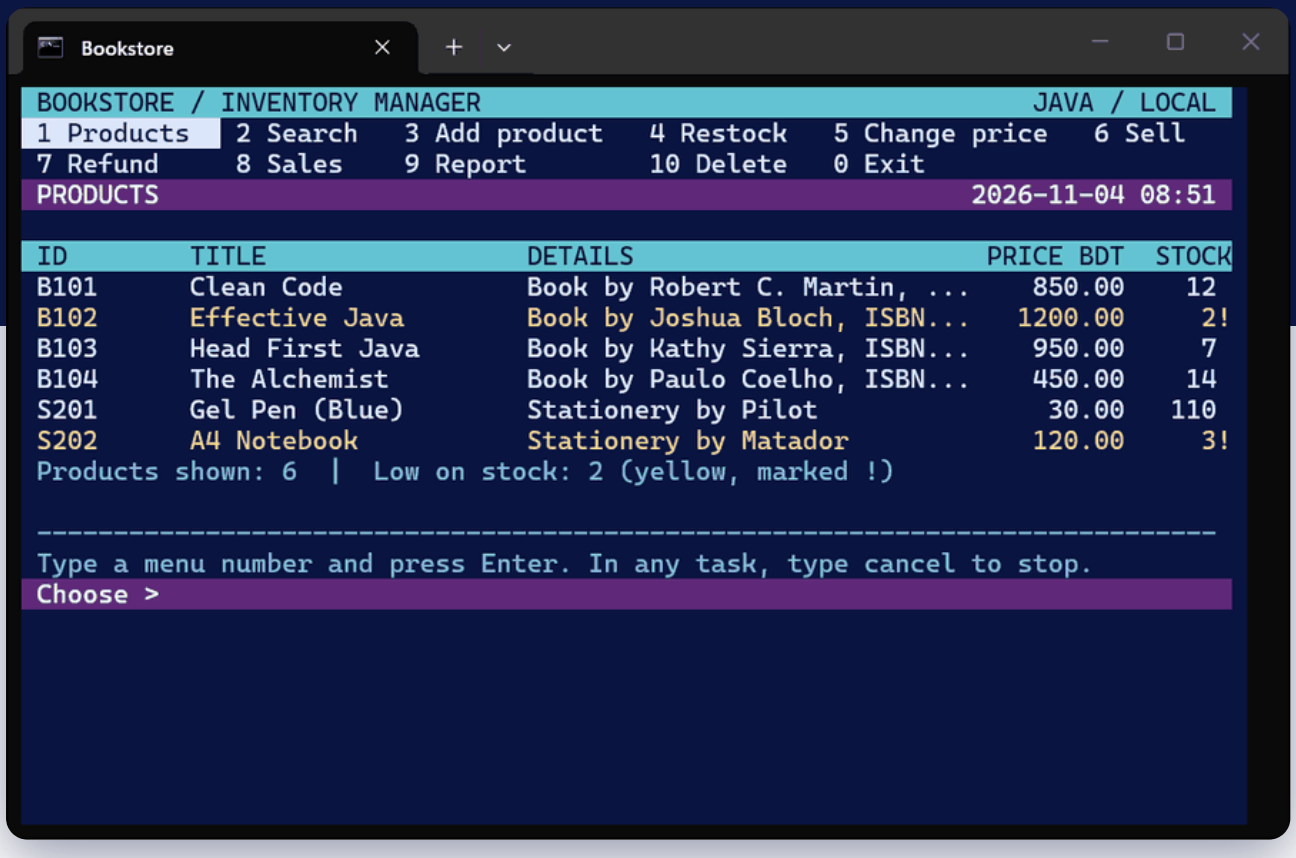
OBJECT-ORIENTED PROGRAMMING · COURSE PROJECT

PROJECT REPORT

Bookstore Inventory Manager

A Java console application for a small bookstore



1 Objective

2 Introduction

3 Methodology

4 Discussion and Conclusion

5 References

LANGUAGE
Java 17

CODE
4 files · 658 lines

DATE
November 2026

Source code: https://github.com/AtlasRoX/Bookstore-JAVA

Contents

1 **Objective** 2

2 **Introduction** 2

2.1 Background 2

2.2 Why object-oriented programming 2

2.3 Features of the program 2

2.4 Organization of this report 3

3 **Methodology** 3

3.1 Tools and development environment 3

3.2 Project structure 4

3.3 System design and program flow 4

3.4 Class design 5

3.5 Implementation of the OOP concepts 6

3.5.1 Abstraction 6

3.5.2 Encapsulation 7

3.5.3 Inheritance 7

3.5.4 Polymorphism 8

3.5.5 Static vs non-static 8

3.5.6 Method overloading 9

3.5.7 Method overriding 10

3.5.8 The this keyword 10

3.5.9 The super keyword 10

3.5.10 Subclass vs superclass reference 11

3.5.11 Diamond problem resolution 11

3.6 Key code behind the features 12

3.6.1 Saving the data 12

3.6.2 Checking every answer 12

3.6.3 Shop rules and errors 13

3.6.4 Deleting a product 13

4 **Discussion and Conclusion** 14

4.1 Results 14

4.2 Discussion 16

4.3 Limitations and future work 17

4.4 Conclusion 17

5 **References** 18

1 Objective

The objective of this project is to design and build a working inventory and sales program for a small bookstore in Java 17, and to use it to apply the object-oriented programming (OOP) concepts taught in the course. In detail, the project aims to:

- O1 Model two product types, books and stationery, that share common data but differ in their details.
- O2 Support the daily work of a shop: adding, restocking and deleting products, changing prices, selling, refunding and reporting.
- O3 Check every answer the user types, so the program never crashes and wrong data is never saved.
- O4 Keep the code short and easy to read: four files, each with one clear job.
- O5 Use all 11 OOP concepts of the course, each one inside a real feature.

2 Introduction

Bookstore Inventory Manager is a Java console application for a small bookstore that sells books and stationery. Staff can add, search, restock and delete products, change prices, sell with an optional 5% member discount, refund sales, and see the sales history and a business report. Every change is saved automatically to the file `bookstore.dat`.

2.1 Background

Many small bookstores still track their stock and sales in a notebook or a spreadsheet. Counting stock by hand, calculating discounts and change, and finding an old sale for a refund are slow and easy to get wrong. This program does that work: it always knows the stock, calculates totals and change, keeps every receipt, and shows the income at a glance.

2.2 Why object-oriented programming

Object-oriented programming builds a program from objects that keep their own data together with the methods that work on that data [1]. A bookstore fits this well. Books and stationery share the same basic data (ID, title, price, stock) but differ in their details (author and ISBN, or brand). That leads directly to an abstract superclass with two subclasses, and from there to inheritance, overriding, polymorphism and the other concepts of the course [1, 2], each one used by a real feature rather than added only for show.

2.3 Features of the program

All work is done from one menu. The user types a key and presses Enter; the menu tabs at the top of every screen show the keys.

Table 2.1 · The ten features and the exit key

Key	Feature	What it does
1	**Products**	Lists every product with ID, title, details, price and stock; low stock (3 or fewer) is highlighted in yellow with "!".
2	**Search**	Finds products by any part of the ID, title, author, brand or ISBN, ignoring upper and lower case.
3	**Add product**	Adds a book (with an ISBN, or "-" if it has none) or a stationery item; an ID that already exists is refused.
4	**Restock**	Adds received units to a product's stock.
5	**Change price**	Sets a new unit price; old receipts keep the price that was paid.
6	**Sell**	Sells one product: checks the quantity against the stock, applies the 5% member discount, checks the cash, shows the change and prints a receipt.
7	**Refund**	Refunds a whole receipt once; the units go back into stock.
8	**Sales**	Shows every receipt with date, product, quantity, total, change and status; refunded receipts are yellow.
9	**Report**	Shows income after refunds, units sold, number of products, low-stock count, units in stock, products added this session and stock value.
10	**Delete**	Removes a product after a yes/no question; its old receipts stay in the sales history.
0	**Exit**	Closes the program; every change was already saved when it was made.

Every question checks its answer: a wrong answer (an unknown ID, too little cash, a letter instead of a number) is explained in a red bar and asked again, and typing `cancel` stops the current task without changing anything.

2.4 Organization of this report

Section 3 (Methodology) explains how the program was built: the tools, the design, and the code of every important part with an explanation. Section 4 (Discussion and Conclusion) shows the finished program in use, checks each objective, describes the problems solved during the work, and lists the limitations and future work. Section 5 lists the references.

3 Methodology

The project was built in four steps: (1) choosing the tools, (2) dividing the program into four classes with one job each, (3) designing the product class hierarchy, and (4) writing the code for every feature, using each OOP concept where it fits naturally. This section follows the same order. All code shown is copied from the project files, with the real line numbers.

3.1 Tools and development environment

Table 3.1 · Tools and technologies

Tool / technology	Used for
Java 17 (JDK: `javac`, `java`)	Writing, compiling and running the program
Java standard library	`ArrayList`, `Scanner` for keyboard input [7], `String.format`, `LocalDate` / `LocalDateTime`
Java object serialization	Saving and loading all products and sales in one step [5, 6]

Tool / technology	Used for
ANSI escape codes	Colors and clearing the screen in the terminal [9]
Windows Terminal + `run.bat`	Opening the program in a window sized exactly 80 × 24 characters [10]
Git and GitHub	Version control; the source code is published at github.com/AtlasRoX/Bookstore-JAVA [11]

3.2 Project structure

Table 3.2 · Source files and their jobs

File	Lines	Job
`Product.java`	126	Interfaces Item, Trackable, Promotable; abstract class Product; subclasses Book and Stationery
`Bookstore.java`	145	Shop rules (add, restock, delete, sell, refund, search), saving and loading; class Sale (one receipt)
`BookstoreUI.java`	224	Everything on screen: colors, menu tabs, tables, receipt, questions
`Main.java`	163	Starts the program, runs the menu and one method per task

How to run: double-click `run.bat`, which compiles the program and opens it in an 80 × 24 window; or, in the project folder, run `javac *.java` and then `java Main`. JDK 17 or newer is required.

3.3 System design and program flow

The program is split into layers, so each class has one job (Figure 3.1). The user only talks to `BookstoreUI`; `Main` decides what happens; `Bookstore` checks the shop's rules and saves; `Product` and `Sale` hold the data. `Bookstore` never prints anything, so the rules do not depend on the screen.

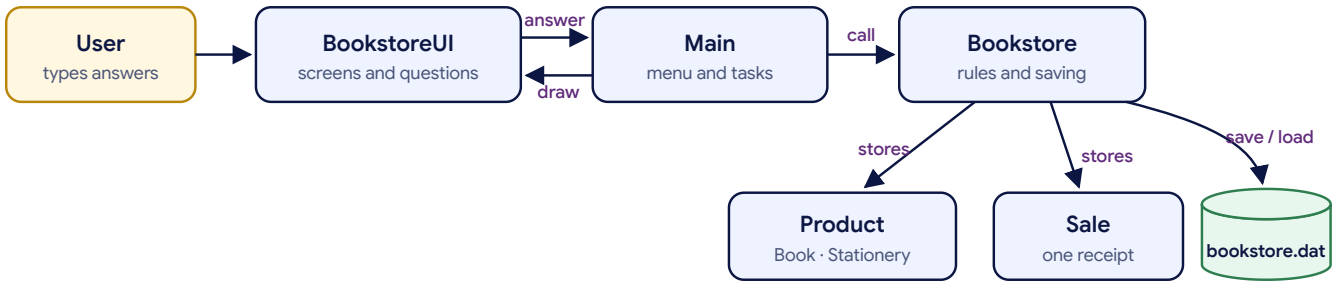


Figure 3.1 · How the parts of the program work together

One run of the program follows these steps:

1. **Start.** `run.bat` compiles the code and opens the window; `main()` loads `bookstore.dat` (or starts an empty shop the first time).
2. **Menu.** The Products screen appears and the menu loop asks "Choose".
3. **Task.** A `switch` in `Main` runs the chosen task; there is one method per menu key.
4. **Questions.** The task asks its questions through `BookstoreUI`; a wrong answer is asked again, and `cancel` stops the task.
5. **Rules and saving.** `Bookstore` checks its rules, changes the data and saves the whole shop to `bookstore.dat` at once.
6. **Result.** The screen is redrawn with the result in green, or a broken rule is shown in red; then the program returns to the menu.

7. **Exit.** Key 0 closes the program. Nothing is lost, because every change was saved the moment it was made.

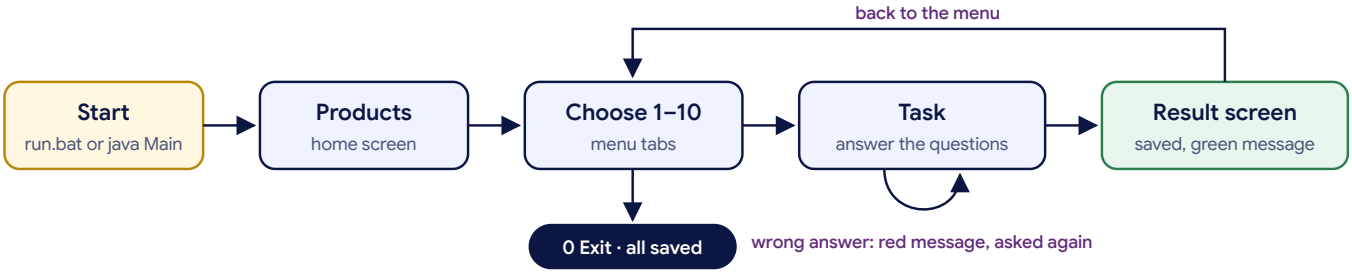


Figure 3.2 · User flow: from start to the result screen and back to the menu

The main method. `main()` carries out this flow. It first sets the US number format, so prices always print as 450.50. It loads the saved shop; if the file is damaged, it shows the error and stops instead of overwriting the file. Then an endless `while` loop shows the menu, and a `switch` runs the method for the chosen key. Every task runs inside one `try` / `catch`: when a rule is broken anywhere, the `RuntimeException` is caught here, shown in red, and the loop simply continues, so the program never crashes.

Main.java lines 8–40

```
8   // Loads the data, then repeats: show the menu, run the chosen task.
9   public static void main(String[] args) {
10      Locale.setDefault(Locale.US); // always show prices as 450.50 (with a dot) on every computer
11      try {
12          store = Bookstore.load();
13      } catch (RuntimeException e) {
14          ui.error(e.getMessage());
15          System.exit(1); // stop with an error code, so run.bat keeps the window open to read this
16      }
17      ui.newScreen(1);
18      ui.showProducts(store.getProducts());
19      while (true) {
20          try {
21              String choice = ui.askMenuChoice();
22              switch (choice) {
23                  case "1": ui.newScreen(1); ui.showProducts(store.getProducts()); break;
24                  case "2": ui.newScreen(2); search(); break;
25                  case "3": ui.newScreen(3); addProduct(); break;
26                  case "4": ui.newScreen(4); restock(); break;
27                  case "5": ui.newScreen(5); changePrice(); break;
28                  case "6": ui.newScreen(6); sell(); break;
29                  case "7": ui.newScreen(7); refund(); break;
30                  case "8": ui.newScreen(8); ui.showSales(store.getSales()); break;
31                  case "9": ui.newScreen(9); ui.showReport(store); break;
32                  case "10": ui.newScreen(10); deleteProduct(); break;
33                  case "0": ui.goodbye(); return;
34                  default: ui.error("Please choose a number from the menu.");
35              }
36          } catch (RuntimeException e) {
37              ui.error(e.getMessage()); // show the problem in red, then go back to the menu
38          }
39      }
40  }
```

3.4 Class design

The product classes form the main hierarchy (Figure 3.3). `Product` is the abstract superclass that holds the data every product has. `Book` and `Stationery` extend it and add only their own fields. `Product` also implements two interfaces, `Trackable` and `Promotable`, which both extend `Item`; this creates the diamond problem solved in 3.5.11. `Sale` is a separate small class: it copies the product's ID, title and total, so an old receipt never changes.

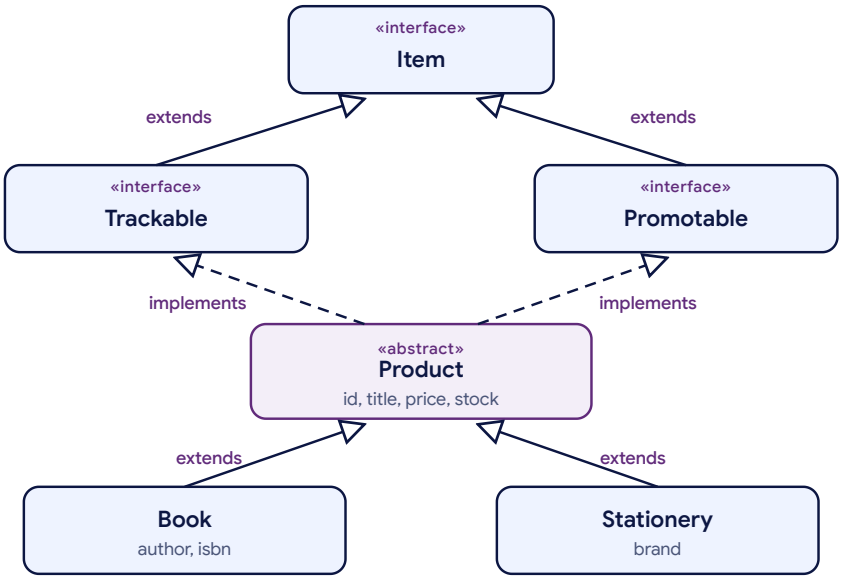


Figure 3.3 · Class hierarchy of the products (solid line: extends, dashed line: implements)

3.5 Implementation of the OOP concepts

Table 3.3 lists the 11 OOP concepts of the course and where each one is used. Sections 3.5.1 to 3.5.11 then explain each concept with its code.

Table 3.3 · The 11 OOP concepts and where they are used

#	Concept	File	In this program
3.5.1	**Abstraction**	Product.java	`abstract class Product` with abstract `getType()` and `getDetails()`
3.5.2	**Encapsulation**	Product, Bookstore	`private` fields; setters refuse negative values
3.5.3	**Inheritance**	Product.java	`Book` and `Stationery` extend `Product`
3.5.4	**Polymorphism**	BookstoreUI.java	`p.getDetails()` runs the Book or the Stationery version
3.5.5	**Static vs non-static**	Product.java	`static CURRENCY`, a `static` block, `getCreatedCount()`
3.5.6	**Method overloading**	Product, Bookstore	two `Book(...)` constructors; two `restock(...)` methods
3.5.7	**Method overriding**	Product.java	`@Override` `getType()`, `getDetails()`, `getCategory()`
3.5.8	**The this keyword**	Product.java	`this.id = id` and `this(..., "N/A")`
3.5.9	**The super keyword**	Product.java	`super(id, title, price, stock)` and `Trackable.super`
3.5.10	**Subclass vs superclass reference**	Main.java	`Book book`, then `Product product = book` (upcasting)
3.5.11	**Diamond problem resolution**	Product.java	two interfaces with the same default `getCategory()`

3.5.1 Abstraction

Product.java

Abstraction means showing only what is needed and hiding the details [1]. `Product` is declared `abstract`, so `new Product(...)` does not compile: the shop can only create a `Book` or a `Stationery`. `getType()` and `getDetails()` have no body here, so every product must be able to describe itself, but each type decides how. The tables, search and

receipts work only with `Product` and never need to know the exact type.

Product.java lines 18–77

```java
// ABSTRACTION: Product is abstract, so the program can only make a Book or a Stationery.
// ENCAPSULATION: the fields are private and are used only through getters and setters.
public abstract class Product implements Trackable, Promotable, Serializable {
…
// ABSTRACT METHODS: every subclass must write its own version (overriding).
public abstract String getType();
public abstract String getDetails();
```

3.5.2 Encapsulation

Product.java · Bookstore.java

Encapsulation means keeping an object's data private and changing it only through its own methods. The fields of `Product` are `private`; other classes read them with getters and change them with setters, which refuse a negative price or stock, so a product can never hold impossible values. `Bookstore` also hides its lists of products and sales, so only its own methods (add, sell, refund …) can change them, and every change is saved.

Product.java lines 32–62

```java
// NON-STATIC (instance) variables: every product object has its own copy.
private String id, title;
private double price;
private int stock;
…
// NON-STATIC METHODS: getters and setters work on one object's own data.
public String getId() { return id; }
public String getTitle() { return title; }
public double getPrice() { return price; }
public int getStock() { return stock; }
// The setters protect the data: a negative value is refused.
public void setPrice(double price) {
    if (price >= 0) this.price = price;
}
public void setStock(int stock) {
    if (stock >= 0) this.stock = stock;
}
```

Bookstore.java lines 8–14

```java
private static final String FILE = "bookstore.dat"; // where everything is saved
private ArrayList<Product> products = new ArrayList<>(); // all products
private ArrayList<Sale> sales = new ArrayList<>(); // all receipts
// Getters for the two lists.
public ArrayList<Product> getProducts() { return products; }
public ArrayList<Sale> getSales() { return sales; }
```

3.5.3 Inheritance

Product.java

Inheritance lets a subclass reuse everything from its superclass [2]. `Book` and `Stationery` `extend` `Product`, so they get id, title, price, stock, all getters and setters and `calculateDiscount()` without writing them again. Each subclass only adds what is special about it: a book has an author and an ISBN, a stationery item has a brand. Because both are a

`Product`, the inventory and every screen treat them the same way.

Product.java lines 80–118

```
80  // INHERITANCE: Book is a subclass of the superclass Product.
81  class Book extends Product {
82      private static final long serialVersionUID = 1L;
83      private String author, isbn;
…
97  public String getAuthor() { return author; } // only a Book has an author
…
107 // INHERITANCE: Stationery is another subclass of Product.
108 class Stationery extends Product {
109     private static final long serialVersionUID = 1L;
110     private String brand;
…
118 public String getBrand() { return brand; } // only Stationery has a brand
```

3.5.4 Polymorphism

BookstoreUI.java

Polymorphism means one call, many forms [2]. In `showProducts` every item is held in a `Product` variable, yet `p.getDetails()` runs Book's version ("Book by …, ISBN …") for books and Stationery's version ("Stationery by …") for stationery: Java chooses the method of the real object while the program runs (runtime polymorphism). The product card does the same with `p.getType()`. Method overloading (3.5.6) is the compile-time form of polymorphism.

BookstoreUI.java lines 136–168

```
136 // A table of products. Low stock (3 or fewer) is yellow and marked with "!".
137 public void showProducts(ArrayList<Product> list) {
138     line(CYAN, String.format(" %-9s %-21s %-29s %9s %6s", "ID", "TITLE", "DETAILS", "PRICE BDT",
    "STOCK"));
139     int low = 0;
140     for (Product p : list) {
141         String color = BODY;
142         String stock = p.getStock() + " ";
143         if (p.getStock() <= 3) {
144             color = YELLOW;
145             stock = p.getStock() + "!";
146             low++;
147         }
148         line(color, String.format(" %-9s %-21s %-29s %9.2f %6s",
149                 fit(p.getId(), 9), fit(p.getTitle(), 21), fit(p.getDetails(), 29), p.getPrice(),
    stock));
150     }
151     if (list.isEmpty()) line(DIM, " No products found. Choose 3 to add one.");
152     line(DIM, " Products shown: " + list.size() + "  |  Low on stock: " + low + " (yellow, marked
    !)");
153     line(BODY, "");
154 }
…
161 // The white product card: type, details, category, price and stock.
162 public void showProduct(Product p) {
163     card(p.getType() + ": " + p.getTitle() + "  (" + p.getId() + ")");
164     card(p.getDetails());
165     card("Category: " + p.getCategory());
166     card(String.format("Price %.2f %s   |   In stock: %d", p.getPrice(), Product.CURRENCY,
    p.getStock()));
167     line(BODY, "");
168 }
```

3.5.5 Static vs non-static

Product.java · BookstoreUI.java · Main.java

A `static` member belongs to the class and is shared by all objects; a non-static member belongs to each object. `CURRENCY` is one shared text, set once in a `static` block when Java loads the class. `createdCount` is a shared counter that every constructor call increases; the static method `Product.getCreatedCount()` reads it and the Report shows it as "New products added this session". `price` and `stock` are non-static: every product has its own. `Main`

uses only static members, so no `Main` object is ever created.

Product.java lines 23–47

```
23  // STATIC variables belong to the class and are shared by every product.
24  public static final String CURRENCY;
25  private static int createdCount = 0; // products created while the program runs (shown in the Report)
27  // STATIC BLOCK: runs once, when Java first loads the Product class.
28  static {
29      CURRENCY = "BDT";
30  }
…
43  createdCount++;
…
46  // STATIC METHOD: called on the class itself, e.g. Product.getCreatedCount()
47  public static int getCreatedCount() { return createdCount; }
```

BookstoreUI.java lines 166–220

```
166 card(String.format("Price %.2f %s   |   In stock: %d", p.getPrice(), Product.CURRENCY, p.getStock()));
…
220 line(BODY, String.format(" %-58s%19d", "New products added this session", Product.getCreatedCount()));
```

Main.java lines 4–6

```
4   public class Main {
5       static BookstoreUI ui = new BookstoreUI(); // draws the screens and reads answers
6       static Bookstore store;                    // the shop's products and sales
```

3.5.6 Method overloading

Product.java · Bookstore.java · Main.java

Method overloading means several methods or constructors with the same name but different parameters; the compiler picks the right one from the arguments. `Book` has two constructors: with an ISBN, and without one (many local books have none, so the user types “-”). `Bookstore` has two `restock` methods: the Restock screen calls `restock(String, int)` with the product ID, which finds the product and calls `restock(Product, int)` to do the work.

Product.java lines 85–93

```
85  // CONSTRUCTOR OVERLOADING 1: a book with an ISBN.
86  public Book(String id, String title, double price, int stock, String author, String isbn) {
…
92  // CONSTRUCTOR OVERLOADING 2: a book without an ISBN. THIS(...) calls the constructor above.
93  public Book(String id, String title, double price, int stock, String author) {
```

Bookstore.java lines 48–58

```
48  // METHOD OVERLOADING: restock by product ID, or by the product object itself.
49  public void restock(String id, int quantity) {
50      restock(get(id), quantity);
51  }
53  // Adds units to the stock and saves.
54  public void restock(Product p, int quantity) {
55      if (quantity < 1) throw new RuntimeException("Quantity must be at least 1.");
56      p.setStock(p.getStock() + quantity);
57      save();
58  }
```

Main.java lines 81–101

```
81  String isbn = ui.askText("ISBN (type - if it has none)");
82  Book book; // SUBCLASS reference
83  if (isbn.equals("-")) book = new Book(id, title, price, stock, author); // overloaded constructor
84  else book = new Book(id, title, price, stock, author, isbn);
…
101 store.restock(p.getId(), ui.askNumber("Quantity to add", 1, 1000000)); // overloaded restock(String,
    int)
```

3.5.7 Method overriding

Product.java

Overriding means a subclass writes its own version of a method it received from its superclass or an interface. `Product` only declares `getType()` and `getDetails()`; `Book` and `Stationery` each give their own body. `@Override` asks the compiler to check that the method really exists in the parent, so a spelling mistake becomes a compile error. These overrides are what make the polymorphism in 3.5.4 work.

Product.java lines 99–125

```
99  // METHOD OVERRIDING: Book's own versions of Product's abstract methods.
100 @Override
101 public String getType() { return "BOOK"; }
103 @Override
104 public String getDetails() { return "Book by " + author + ", ISBN " + isbn; }
…
121 @Override
122 public String getType() { return "STATIONERY"; }
124 @Override
125 public String getDetails() { return "Stationery by " + brand; }
```

3.5.8 The this keyword

Product.java

`this` means “the current object”. In `this.id = id` the left side is the object’s own field and the right side is the constructor parameter; without `this` the parameter would hide the field. `this(...)` calls another constructor of the same class: the Book constructor without an ISBN passes its values to the full constructor and fills the ISBN with “N/A”, so the setup code is written only once. The `Sale` constructor in `Bookstore.java` uses `this` the same way.

Product.java lines 37–95

```
37  // Constructor: stores the values and counts the new product.
38  public Product(String id, String title, double price, int stock) {
39      this.id = id; // THIS: this.id is the object's field, id is the parameter
40      this.title = title;
41      this.price = price;
42      this.stock = stock;
43      createdCount++;
44  }
…
92  // CONSTRUCTOR OVERLOADING 2: a book without an ISBN. THIS(...) calls the constructor above.
93  public Book(String id, String title, double price, int stock, String author) {
94      this(id, title, price, stock, author, "N/A");
95  }
```

3.5.9 The super keyword

Product.java

`super` refers to the parent class. A subclass constructor starts with `super(id, title, price, stock)`, which runs the `Product` constructor first: it stores the shared fields and increases the static counter. Only then does the subclass store its own fields (author and ISBN, or brand). The other form, `Trackable.super.getCategory()`, calls a parent interface’s method; see 3.5.11.

Product.java lines 85–116

```
85  // CONSTRUCTOR OVERLOADING 1: a book with an ISBN.
86  public Book(String id, String title, double price, int stock, String author, String isbn) {
87      super(id, title, price, stock); // SUPER: runs the Product constructor first
88      this.author = author;
89      this.isbn = isbn;
90  }
…
112 // Constructor: the Product part is filled first by super(...).
113 public Stationery(String id, String title, double price, int stock, String brand) {
114     super(id, title, price, stock); // SUPER: runs the Product constructor first
115     this.brand = brand;
116 }
```

3.5.10 Subclass vs superclass reference

Main.java

A subclass reference (`Book book`) can use everything a Book has, including `getAuthor()`. A superclass reference (`Product product`) can point to any subclass object (upcasting) but only sees the methods declared in `Product`. In Add product the new object is first held as a `Book` or `Stationery` to read its author or brand; then `product = book` stores it as a `Product`. From then on every item is treated the same: `store.add(product)` puts it in the `ArrayList<Product>`, and `product.getType()` still runs Book's own version.

Main.java lines 66–94

```
66  // 3 Add product: asks every detail, then saves a Book or a Stationery.
67  static void addProduct() {
68      char type = ui.askLetter("Type: B = book, S = stationery", "BS");
69      String id = ui.askText("ID (short code, e.g. B1)");
70      while (store.find(id) != null) {
71          ui.error("ID " + id + " is already used. Please choose another.");
72          id = ui.askText("ID (short code, e.g. B1)");
73      }
74      String title = ui.askText("Title");
75      double price = ui.askMoney("Price (BDT)", 0);
76      int stock = ui.askNumber("Stock", 0, 1000000);
77      Product product; // SUPERCLASS reference: can hold a Book or a Stationery
78      String extra;
79      if (type == 'B') {
80          String author = ui.askText("Author");
81          String isbn = ui.askText("ISBN (type - if it has none)");
82          Book book; // SUBCLASS reference
83          if (isbn.equals("-")) book = new Book(id, title, price, stock, author); // overloaded
    constructor
84          else book = new Book(id, title, price, stock, author, isbn);
85          extra = " by " + book.getAuthor(); // getAuthor() works only through the Book reference
86          product = book;                    // UPCASTING: the Book is stored as a Product
87      } else {
88          Stationery item = new Stationery(id, title, price, stock, ui.askText("Brand"));
89          extra = " from " + item.getBrand();
90          product = item;
91      }
92      store.add(product);
93      showResult(3, "Added " + product.getType() + " \"" + title + "\"" + extra + ".");
94  }
```

3.5.11 Diamond problem resolution

Product.java · BookstoreUI.java

Java does not allow `class C extends A, B`, which avoids the diamond problem for classes. With interfaces it can still happen through default methods [3]: `Trackable` and `Promotable` both extend `Item` and both give their own `default getCategory()`. `Product` implements both, so Java cannot know which one to use and refuses to compile until `Product` overrides the method [4]. It chooses explicitly with `Trackable.super.getCategory()` and `Promotable.super.getCategory()`; the result, "Tracked stock & Promotable", is shown on the product card (Figures 4.4 and 4.6).

Product.java lines 4–16

```
4   interface Item {
5       default String getCategory() { return "Item"; }
6   }
8   interface Trackable extends Item {
9       @Override
10      default String getCategory() { return "Tracked stock"; }
11  }
13  interface Promotable extends Item {
14      @Override
15      default String getCategory() { return "Promotable"; }
16  }
```

Product.java lines 20–73

```java
public abstract class Product implements Trackable, Promotable, Serializable {
...
// DIAMOND PROBLEM SOLVED: we say exactly which parent versions to use.
@Override
public String getCategory() {
    return Trackable.super.getCategory() + " & " + Promotable.super.getCategory();
}
```

BookstoreUI.java line 165

```java
card("Category: " + p.getCategory());
```

3.6 Key code behind the features

Four more parts of the code make the program safe to use: saving, checking every answer, the shop's rules, and deleting a product without losing its history.

3.6.1 Saving the data

Bookstore.java

All products and sales live inside one `Bookstore` object. Because the classes implement `Serializable` [5], `save()` writes the whole object to `bookstore.dat` with one `ObjectOutputStream.writeObject` call [6] after every change, and `load()` reads it back when the program starts. If the file is damaged, the program says so and never overwrites it.

Bookstore.java lines 107–124

```java
// Saves the whole bookstore (products and sales) to the file in one step.
private void save() {
    try (ObjectOutputStream out = new ObjectOutputStream(new FileOutputStream(FILE))) {
        out.writeObject(this);
    } catch (IOException e) {
        throw new RuntimeException("Could not save: " + e.getMessage());
    }
}
// Reads the saved bookstore, or starts an empty one the first time.
public static Bookstore load() {
    if (!new File(FILE).exists()) return new Bookstore();
    try (ObjectInputStream in = new ObjectInputStream(new FileInputStream(FILE))) {
        return (Bookstore) in.readObject();
    } catch (Exception e) {
        throw new RuntimeException("Could not read " + FILE + ": " + e.getMessage());
    }
}
```

3.6.2 Checking every answer

BookstoreUI.java

Every question goes through `askText`, which reads one line with a `Scanner` [7]: it draws the purple input bar, ignores empty answers and turns `cancel` into an exception that ends the task. `askNumber` (and `askMoney`) keep asking until

the answer is a number in the allowed range, so a mistake never cancels the whole task.

BookstoreUI.java lines 83–94

```java
// Keeps asking until something is typed. Typing "cancel" stops the current task.
public String askText(String question) {
    while (true) {
        // Draw a purple bar, then \r jumps back so the typing starts right after the question.
        System.out.print(PURPLE + String.format(" %-78s", question + " >") + "\r " + question + " >
");
        if (!in.hasNextLine()) System.exit(0); // input was closed
        String answer = in.nextLine().trim();
        System.out.print(BODY);
        if (answer.equalsIgnoreCase("cancel")) throw new RuntimeException("Cancelled. Nothing
changed.");
        if (!answer.isEmpty()) return answer;
    }
}
```

BookstoreUI.java lines 96–107

```java
// Asks again until the answer is a whole number from min to max.
public int askNumber(String question, int min, int max) {
    while (true) {
        try {
            int number = Integer.parseInt(askText(question));
            if (number >= min && number <= max) return number;
        } catch (NumberFormatException e) {
            // not a number, so ask again
        }
        error("Please enter a whole number from " + min + " to " + max + ".");
    }
}
```

3.6.3 Shop rules and errors

Bookstore.java

`Bookstore` checks its rules again before changing any data: the quantity must be in stock, the cash must cover the total, and members get 5% off through `calculateDiscount(5)`. A broken rule throws a `RuntimeException` with a clear message; the `catch` in the menu loop (3.3) shows it in a red bar and returns to the menu.

Bookstore.java lines 72–79

```java
// Members get 5% off. The total is rounded to 2 decimal places.
public double total(Product p, int quantity, boolean member) {
    if (quantity < 1 || quantity > p.getStock())
        throw new RuntimeException("Quantity must be from 1 to " + p.getStock() + ".");
    double total = p.getPrice() * quantity;
    if (member) total = total - p.calculateDiscount(5) * quantity;
    return Math.round(total * 100) / 100.0;
}
```

3.6.4 Deleting a product

Main.java · Bookstore.java

Delete (key 10) first shows the product card and asks a yes/no question, so a product is never removed by mistake. `Bookstore.delete()` then removes it from the list and saves. Old receipts are not touched: every `Sale` keeps its own copy of the title and the total, so the sales history and the income in the report stay correct (Figures 4.11 and 4.12). A

receipt of a deleted product can still be refunded: `refund()` looks the product up with `find()`, which returns `null` instead of stopping with an error, so the money is always given back and the stock is returned only if the product still exists. The refund message tells the user which case happened.

Main.java lines 147–162

```
147  String stock = (store.find(sale.productId) != null) ? "Stock returned." : "No stock returned (product
     deleted).";
...
151  // 10 Delete: removes a product after asking to be sure. Old receipts are kept.
152  static void deleteProduct() {
153      ui.showProducts(store.getProducts());
154      Product p = askProduct();
155      ui.showProduct(p);
156      if (!ui.askYes("Delete this product")) {
157          ui.message("Not deleted. Nothing changed.");
158          return;
159      }
160      store.delete(p);
161      showResult(10, "Deleted \"" + p.getTitle() + "\". Its old receipts are kept.");
162  }
```

Bookstore.java lines 66–105

```
66   // Removes a product and saves. Its old receipts stay, because a Sale keeps its own title and total.
67   public void delete(Product p) {
68       products.remove(p);
69       save();
70   }
...
98   // Marks the receipt refunded and puts the units back in stock (if the product was not deleted).
99   public void refund(Sale sale) {
100      if (sale.refunded) throw new RuntimeException("Receipt #" + sale.id + " was already refunded.");
101      sale.refunded = true;
102      Product p = find(sale.productId); // null if the product was deleted
103      if (p != null) p.setStock(p.getStock() + sale.quantity);
104      save();
105  }
```

4 Discussion and Conclusion

4.1 Results

The finished program was run in its own 80 × 24 window, opened by `run.bat`, with a sample catalogue of four books and two stationery items. Figures 4.1 to 4.12 show every feature working, including the checks on wrong answers.

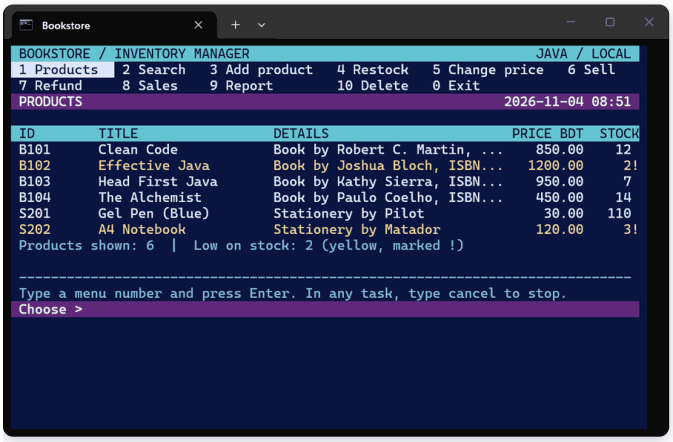


Figure 4.1 · 1 Products: the home screen, with every product, its details, price and stock. Low stock (3 or fewer) is yellow with "!".

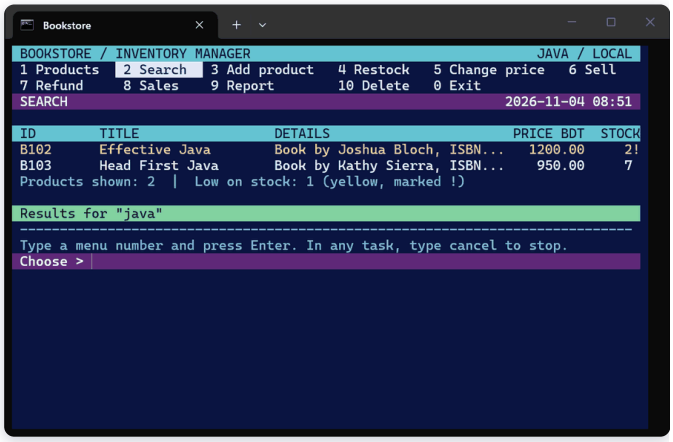


Figure 4.2 · 2 Search: finds products by any part of the ID, title, author, brand or ISBN.

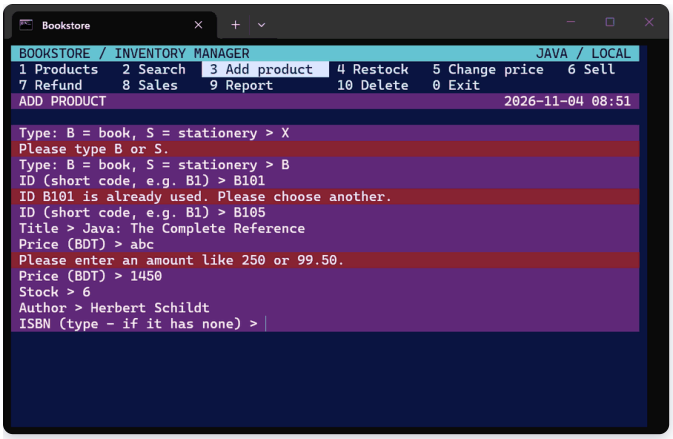


Figure 4.3 · 3 Add product: one question at a time; each wrong answer is explained in red and asked again.

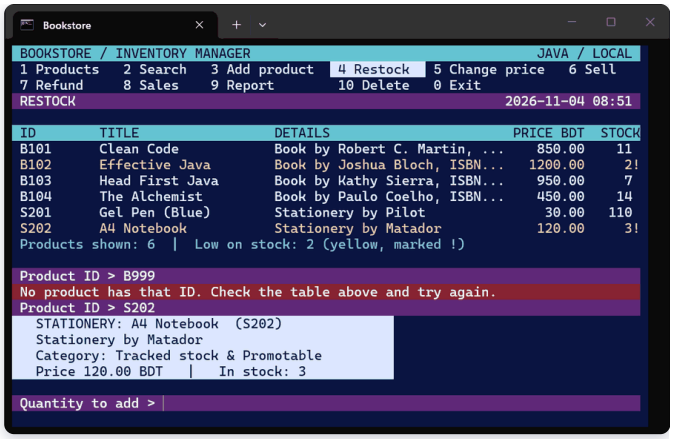


Figure 4.4 · 4 Restock: an unknown ID is refused; the card shows type, details, category and current stock.

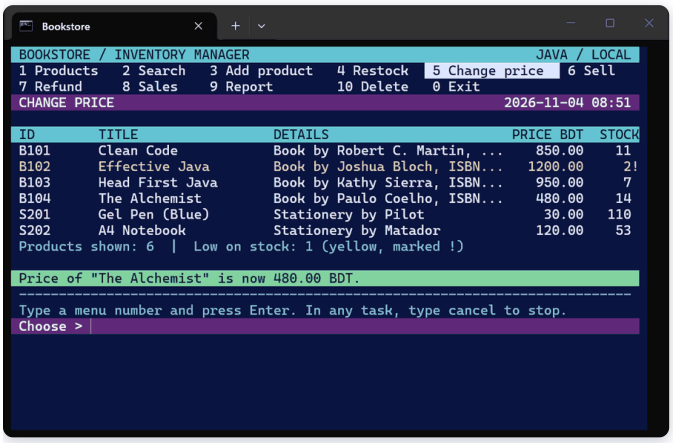


Figure 4.5 · 5 Change price: the new price is saved and shown in the table with a green message.

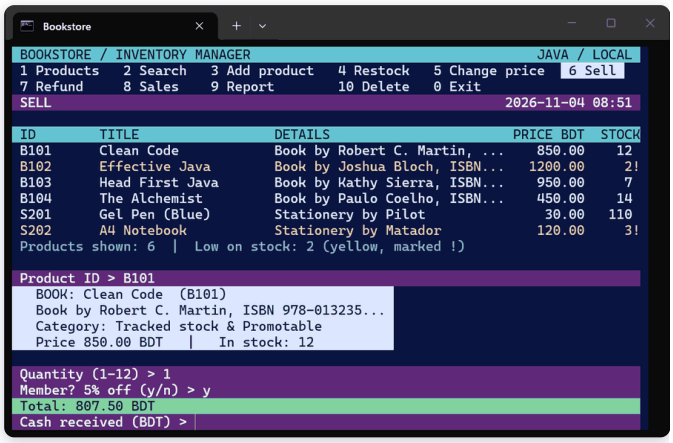


Figure 4.6 · 6 Sell: the quantity range is shown, a member gets 5% off, and the total appears in green.

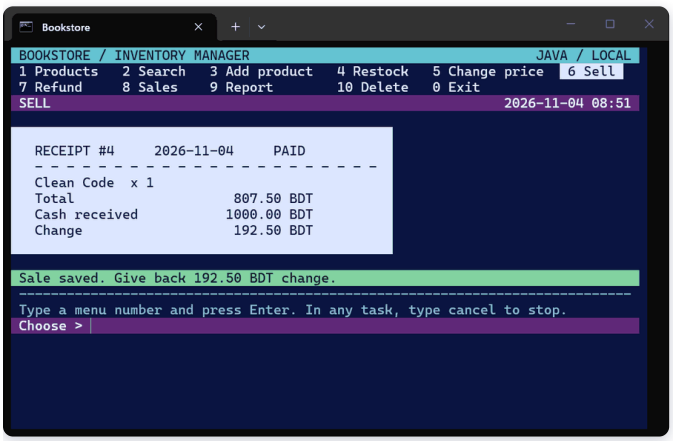


Figure 4.7 · 6 Sell: after payment the screen shows the receipt and the change to give back.

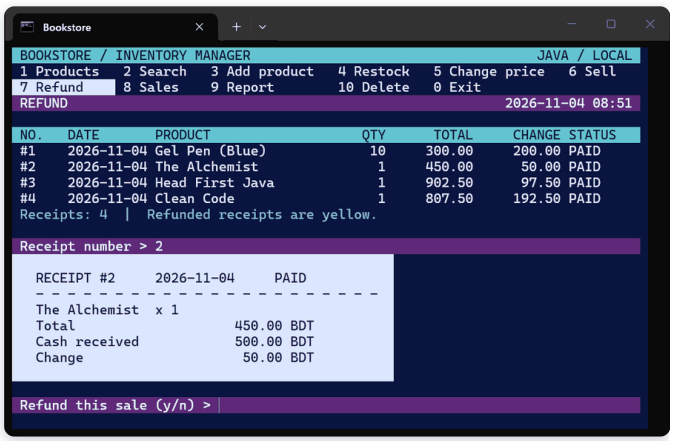


Figure 4.8 · 7 Refund: choose a receipt and confirm; the units go back into stock.

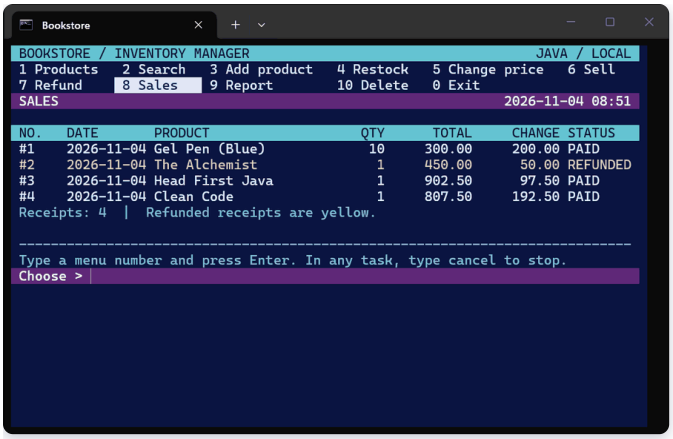


Figure 4.9 · 8 Sales: every receipt with total, change and status; refunded ones are yellow.

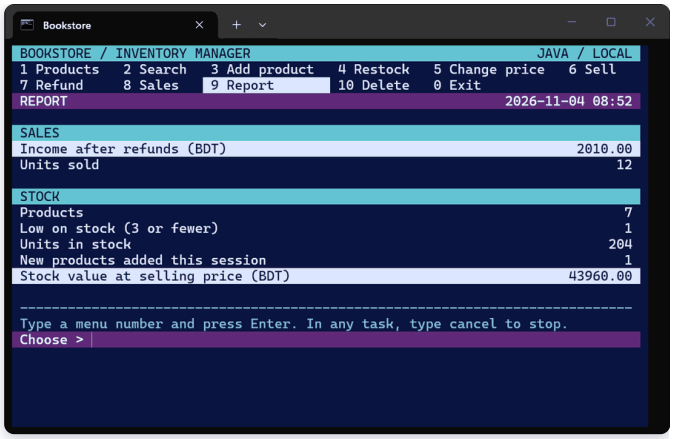


Figure 4.10 · 9 Report: income after refunds, units sold, stock, products added this session and stock value.

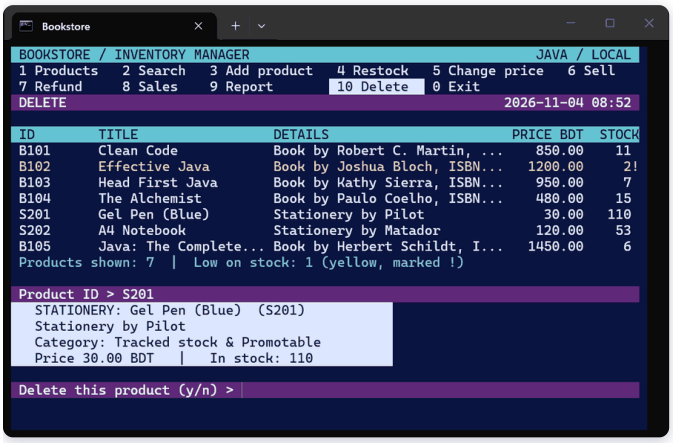


Figure 4.11 · 10 Delete: the product card is shown and the program asks before deleting anything.

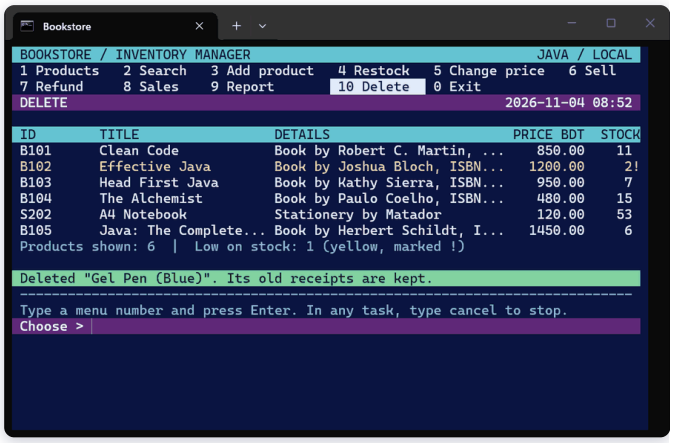


Figure 4.12 · 10 Delete: "Gel Pen (Blue)" is gone from the table; its old receipt stays in the sales history.

4.2 Discussion

Table 4.1 checks each objective from Section 1 against the finished program. All five objectives were met.

Table 4.1 · How each objective was met

	Objective	How it was met	See	Result
O1	**Two product types**	Abstract `Product` with the subclasses `Book` and `Stationery`; one `ArrayList<Product>` holds both	3.4, 3.5.3	**Met**
O2	**Daily shop work**	Ten menu features, with one method per task in `Main`	Figures 4.1–4.12	**Met**
O3	**Never crash, no wrong data**	Questions repeat until the answer is valid; `Bookstore` checks its rules again; one `catch` in the menu loop; setters refuse negative values	3.3, 3.6	**Met**
O4	**Short, readable code**	Four files with one job each, 658 lines in total, with a one-line comment above each part	3.2	**Met**
O5	**All 11 OOP concepts**	Every concept is used by a real feature	3.5, Table 3.3	**Met**

Separating the screen from the rules. `BookstoreUI` is the only class that prints or reads, and `Bookstore` never prints. Because of this, the shop's rules do not depend on the console, and a graphical interface could replace it later without changing `Bookstore` or `Product`.

Saving with serialization. Writing the whole `Bookstore` object with `ObjectOutputStream` [6] needs only a few lines and saves products and sales together in one step. The cost is that the file format depends on the classes; a fixed `serialVersionUID` keeps old files readable after small code changes [5]. Java serialization is also not safe for files from untrusted sources [8]. This program only reads its own file, but a larger system should use a database instead.

Errors as exceptions. Every broken rule throws a `RuntimeException` with a clear message, and one `catch` in the menu loop shows it. All errors therefore look the same to the user, and no rule can crash the program.

Table 4.2 lists the main problems met during the work and how each was solved.

Table 4.2 · Problems and solutions

Problem	Solution
A letter typed where a number was expected would throw `NumberFormatException` and stop the program.	`askNumber` and `askMoney` catch it and ask the same question again.
Changing a price later would change the totals of old receipts.	`Sale` copies the title and the total at the moment of the sale.
On computers with other regional settings, prices could print with a comma (450,50).	`Locale.setDefault(Locale.US)` at start-up, so prices always print as 450.50.
The screens need 80 columns; in a smaller window the tables wrap and break.	`run.bat` opens Windows Terminal at exactly 80 × 24 characters.
A saved file could become unreadable after a small change to a class.	Every saved class declares `serialVersionUID = 1L`.
After a product was deleted, refunding one of its old receipts would stop with an error, because the refund looked the product up with `get()`.	The refund uses `find()`: the money is always given back, and the stock is returned only if the product still exists.
Two interfaces with the same default method do not compile together.	`Product` overrides `getCategory()` and calls both versions with `X.super`.

4.3 Limitations and future work

Table 4.3 · Limitations and possible improvements

Current limitation	Possible improvement
One product per sale	A shopping cart with several products on one receipt
A product's title, author or brand cannot be changed	An edit screen for all product details
Only one copy of the program should use the data file at a time	A database (for example SQLite) that supports several users
The Windows console shows only English letters reliably	A graphical interface (JavaFX) with full Unicode support
No user accounts	Staff login with different permissions
Reports only on screen	Export of reports and receipts to PDF or Excel

4.4 Conclusion

The Bookstore Inventory Manager meets its objectives. It handles the daily work of a small shop (catalogue, stock, sales, refunds and reports) with clear screens, helpful messages and automatic saving. It is short (658 lines in four files) and uses simple Java, yet all 11 OOP concepts of the course appear inside real features: abstraction and inheritance in the product hierarchy, encapsulation in the protected fields, runtime polymorphism in the product table, overloading in the Book constructors and the restock methods, overriding in `getType`, `getDetails` and `getCategory`, static members in the currency and the session counter, `this` and `super` in the constructors, upcasting when a product is added, and the interface diamond problem in the product category. The project showed that these concepts are not only theory: each one made the program shorter, safer or easier to change.

5 References

[1] Oracle, “Lesson: Object-Oriented Programming Concepts,” *The Java Tutorials*. https://docs.oracle.com/javase/tutorial/java/concepts/

[2] Oracle, “Lesson: Interfaces and Inheritance,” *The Java Tutorials*. https://docs.oracle.com/javase/tutorial/java/landl/

[3] Oracle, “Default Methods,” *The Java Tutorials*. https://docs.oracle.com/javase/tutorial/java/landl/defaultmethods.html

[4] J. Gosling et al., *The Java Language Specification, Java SE 17 Edition*, §8.4.8.4 “Inheriting Methods with Override-Equivalent Signatures,” Oracle, 2021. https://docs.oracle.com/javase/specs/jls/se17/html/jls-8.html#jls-8.4.8.4

[5] Oracle, “Interface Serializable,” *Java SE 17 & JDK 17 API Documentation*. https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/io/Serializable.html

[6] Oracle, “Class ObjectOutputStream,” *Java SE 17 & JDK 17 API Documentation*. https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/io/ObjectOutputStream.html

[7] Oracle, “Class Scanner,” *Java SE 17 & JDK 17 API Documentation*. https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/Scanner.html

[8] J. Bloch, *Effective Java*, 3rd ed. Boston, MA, USA: Addison-Wesley, 2018, ch. 12, “Serialization.”

[9] Ecma International, *ECMA-48: Control Functions for Coded Character Sets*, 5th ed., 1991. https://ecma-international.org/publications-and-standards/standards/ecma-48/

[10] Microsoft, “Windows Terminal command line arguments,” *Microsoft Learn*. https://learn.microsoft.com/en-us/windows/terminal/command-line-arguments

[11] Bookstore-JAVA project source code, GitHub. https://github.com/AtlasRoX/Bookstore-JAVA